

TP — Calibrage d'une caméra

1. Objectif du calibrage

Le calibrage sert d'abord à **corriger les distorsions de l'objectif** (radiale et tangentielle) afin d'obtenir des images géométriquement fidèles. Il permet aussi d'**établir une relation précise entre les points 3D du monde et leurs projections 2D** dans l'image en estimant la **matrice intrinsèque** de la caméra, sa **pose** (rotation et translation) et les **coefficients de distorsion**. Grâce à ces paramètres, on peut par exemple mesurer des dimensions sur l'image, estimer la pose d'objets, faire de la réalité augmentée, rectifier des paires stéréo et améliorer les reconstructions 3D.

2. Modèle sténopé : paramètres intrinsèques et extrinsèques

Principe général

Le **modèle sténopé (pinhole)** décrit la projection d'un point 3D (X, Y, Z) du monde sur un point image (u, v) via la relation :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R} \quad \mathbf{t}] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

où :

- $\mathbf{K}$: matrice *intrinsèque*
- $[\mathbf{R} \mid \mathbf{t}]$: *paramètres extrinsèques* (pose de la caméra).

Paramètres intrinsèques

La matrice intrinsèque s'écrit :

$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

- f_x, f_y : focales exprimées **en pixels** (elles peuvent différer si les pixels ne sont pas carrés)
- c_x, c_y : coordonnées du **centre principal** de l'image (en pixels)

Ces paramètres décrivent les **caractéristiques internes** de la caméra et ne dépendent pas de la scène observée.

Paramètres extrinsèques

- $\mathbf{R}$: matrice de **rotation**, orientant la caméra dans le repère monde
- $\mathbf{t}$: vecteur de **translation**, donnant la position du centre optique dans le repère monde.

Ces paramètres définissent la **pose** de la caméra par rapport au monde réel.

Distorsions optiques

Le calibrage estime aussi les coefficients de :

- **Distorsion radiale** : $k_1, k_2, k_3, \dots$
- **Distorsion tangentielle** : p_1, p_2

Ces coefficients permettent de **corriger les déformations** visibles sur les bords de l'image.

3. Lancement de l'exemple (réalisé en Python/OpenCV)

J'ai d'abord consulté l'exemple MATLAB (principes et sorties attendues), puis j'ai **porté le workflow en Python** avec mon script `calibrate.py`.

Chargement et affichage des images

J'ai listé mes images de damier et affiché la première (pour contrôle visuel) :

```
In [ ]: files = list_images("./images_calib")
img = cv2.imdecode(np.fromfile(files[0], np.uint8)
cv2.IMREAD_COLOR)
cv2.imshow("img", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Cela m'a permis d'afficher la première image du damier. En modifiant l'index (files[1], files[2], etc.), j'ai pu visualiser d'autres points de vue.

Pour obtenir la taille d'une image (utile pour vérifier la cohérence du dataset) :

```
In [ ]: (h, w) = img.shape[:2]; print((w, h))
```

Génération des points du damier (world points)

J'ai utilisé la fonction Python équivalente (ma fonction build_obj_pts) pour générer les points 3D idéaux dans le repère du motif :

```
In [ ]: obj_pts_one = build_obj_pts(cols=9, rows=6, sq_mm=25.0)
```

Ces coordonnées sont dans le plan $Z=0$ et en millimètres si sq_mm est en mm.

Calibration automatique

J'ai lancé mon pipeline de calibration (détection robuste + calibration OpenCV) :

```
In [ ]: res = calibrate_fast("./images_calib", cols=9, rows=6, sq_mm=25.0)
```

La structure res contient : K (intrinsèques), dist (coeffs de distorsion), rvecs/tvecs (extrinsèques par vue), per_view (RMS par image), img_pts/obj_pts, etc.

4. Analyse des résultats du calibrage

1. Variable worldPoints

En Python, j'utilise build_obj_pts (équivalent de generateCheckerboardPoints). Elle retourne les coordonnées idéales des coins du damier dans le repère du motif (plan $Z = 0$), espacés de squareSize millimètres.

```
In [ ]: obj_pts_one = build_obj_pts(cols=9, rows=6, sq_mm=25.0) # mm
```

2. Variables boardSize et squareSize

- boardSize : **nombre de coins intérieurs** du damier (lignes × colonnes) ici (9 x 6)
- squareSize : **taille d'un carré** du damier (exprimée en mm) ici (25 mm)
- Les coordonnées (worldPoints) sont exprimées dans le **repère du damier**.

3. Variable imagePoints

En Python/OpenCV, ce sont les points détectés img_pts (sub-pixel) pour chaque image :

```
In [ ]: len(res["img_pts"]), res["img_pts"][0].shape # nb de vues et nb de points dans La
```

4. Nombre d'images utilisées

Le nombre de vues effectivement retenues par la calibration :

```
In [ ]: len(res["files_used"])
```

Dans mon exécution, 10 images ont été utilisées

5. Visualisation des poses

OpenCV ne fournit pas `showExtrinsics` comme MATLAB, mais j'ai :

- les vecteurs de rotation (Rodrigues) `res["rvecs"]` ;
- les vecteurs de translation `res["tvecs"]` (en mm).

J'ai parcouru ces poses pour inspection :

```
In [ ]: for i, (rv, tv) in enumerate(zip(res["rvecs"], res["tvecs"]), 1):  
        print(f"View {i:02d}: rvec={rv.ravel()}, tvec={tv.ravel()} mm")
```

6. Erreurs de reprojection

J'affiche la RMS globale et la RMS par vue (déjà calculées par mon script) :

```
In [ ]: print("RMS global (px):", res["rms"])  
        for pv in res["per_view"]:  
            print(os.path.basename(pv["file"]), "->", f'{pv["err"]:.3f}', "px")
```

- Unité : pixel ;
- Valeur moyenne observée : ~ 0,2 px (bon calibrage, répartition homogène).

7. Focale, point principal et précision

Récupération des intrinsèques :

```
In [ ]: K = res["K"]; fx, fy, s, cx, cy = K[0,0], K[1,1], K[0,1], K[0,2], K[1,2]; print(fx,
```

8. Résultats

=== Camera Calibration ===

Images utilisées : 10 / 10

RMS global : 0.8800 px
Image size : 3024 × 4032 px
Damier (détecté): 6×8 — carré = 28.0 mm

Intrinsic matrix K :
[[3.05384e+03 0.00000e+00 1.51209e+03]
[0.00000e+00 3.06041e+03 2.04822e+03]
[0.00000e+00 0.00000e+00 1.00000e+00]]

$f_x = 3053.83924$ px, $f_y = 3060.40846$ px
 $c_x = 1512.08926$ px, $c_y = 2048.21743$ px
skew (s) = 0.000000
aspect ratio $f_y/f_x = 1.002151$

Distorsion radiale (k1..k6) : [1.681684e-01, -7.179433e-01, 1.377586e+00, nan, nan, nan]
Distorsion tangentielle (p1, p2) : [0.000000e+00, 0.000000e+00]
Coefficients complets (OpenCV) : [1.681684e-01, -7.179433e-01, 0.000000e+00, 0.000000e+00, 1.377586e+00]

Erreurs par image (RMS, px) :

1. IMG_3739.jpeg : 0.692
2. IMG_3745.jpeg : 0.680
3. IMG_3744.jpeg : 0.595
4. IMG_3737.jpeg : 0.756
5. IMG_3736.jpeg : 0.676
6. IMG_3743.jpeg : 1.243
7. IMG_3738.jpeg : 0.579
8. IMG_3746.jpeg : 0.676
9. IMG_3747.jpeg : 1.470
10. IMG_3748.jpeg : 0.959

Rotation vectors (Rodrigues, radians) et Translation vectors (mm) :
View 01 rvec = [-0.081600, 0.014240, 0.005035] tvec = [-71.771, -125.052, 294.882]
View 02 rvec = [-0.097599, 0.008971, 0.003329] tvec = [-73.449, -132.265, 326.577]
View 03 rvec = [0.246883, 0.018469, 0.017325] tvec = [-76.954, -111.926, 319.236]
View 04 rvec = [-0.066287, 0.015734, 0.000332] tvec = [-69.915, -111.599, 363.558]
View 05 rvec = [-0.109315, -0.003052, -0.000672] tvec = [-61.688, -126.890, 368.116]
View 06 rvec = [-0.706444, -0.001477, 0.003769] tvec = [-65.435, -66.097, 398.601]
View 07 rvec = [-0.115202, 0.025624, 0.010650] tvec = [-63.330, -98.967, 430.779]
View 08 rvec = [-0.123363, 0.003792, -0.010526] tvec = [-73.318, -83.322, 454.540]

View 09 rvec = [-0.745528, 0.299343, 0.622536] tvec = [24.154, -128.533, 474.612]
View 10 rvec = [-0.818064, -0.383665, -0.610075] tvec = [-127.809, -64.960, 395.976]

9. Commentaires sur les résultats

Le calibrage repose sur **10/10 vues**, offrant une base suffisante et variée.

L'erreur **RMS globale de 0,88 px** indique une qualité correcte

La **taille 3024 × 4032 px** et la **matrice intrinsèque** ($f_x \approx 3053,84 \text{ px}$, $f_y \approx 3060,41 \text{ px}$; $c_x \approx 1512,09 \text{ px}$, $c_y \approx 2048,22 \text{ px}$; $skew = 0$; $f_y/f_x \approx 1,002$) sont cohérentes avec le capteur de mon smartphone (Iphone 12) en mode portrait car les pixels sont quasi carrés selon le document constructeur.

La **distorsion radiale** est modérée ($k1 > 0$, $k2 < 0$, $k3 > 0$) et la **tangentielle négligeable** ($p1 = p2 = 0$), signe d'un **bon centrage optique**.

Les **erreurs par image** restent globalement faibles à l'exception de 06 et 09.

Les **poses** couvrent à peu près **0,30 à 0,48 m** de distance, avec des inclinaisons marquées sur **06/09/10**, apportant une diversité angulaire utile.

Conclusion : le calibrage est exploitable tel quel.

